

# Braq Connect™

Braq Uni — WhatsApp Uniform Platform

## Platform Technical Documentation

Version 1.0 | Prepared: 2 July 2026

### PLATFORM TYPE

WhatsApp-Based Uniform Manufacturing & Retail

### BACKEND

Node.js v22 · Express.js · ESM Modules

### DATABASE

PostgreSQL (Supabase) · 21 Migrations

### AI ENGINE

Google Gemini 2.5-flash

### MESSAGING

Meta WhatsApp Cloud API v20.0

### DASHBOARD

React 18 · Vite · Ant Design

### PDF ENGINE

PDFKit

### AUTH

JWT (HS256) · bcryptjs · Role-based access

### DEPLOYMENT

Render.com · npm workspaces monorepo



# Table of Contents

|            |                                   |
|------------|-----------------------------------|
| <b>1.</b>  | Executive Summary                 |
| <b>2.</b>  | Architecture Overview             |
| <b>3.</b>  | Technology Stack                  |
| <b>4.</b>  | Database Schema                   |
| <b>5.</b>  | WhatsApp Bot — Conversation Flows |
| <b>6</b>   | API Endpoints                     |
| <b>7</b>   | Staff Dashboard                   |
| <b>8</b>   | School Catalog                    |
| <b>9</b>   | Quotation System                  |
| <b>10.</b> | AI Integration                    |
| <b>11.</b> | Security & Authentication         |
| <b>12.</b> | Deployment & Configuration        |
| <b>13.</b> | Roles & Permissions               |

# 1. Executive Summary

Braq Connect™ (also referred to as Braq Uni) is a fully integrated, WhatsApp-first business platform purpose-built for a South African uniform manufacturing and retail company. The platform enables clients to browse products, request quotations, track orders, log support tickets, and communicate with consultants — all through WhatsApp without installing any additional app.

On the staff side, a React-based web dashboard allows consultants, managers, and administrators to manage orders, quotations, clients, purchase orders, support tickets, broadcasts, and a live consultant inbox connected directly to the WhatsApp conversations.

## Business Context

- Target market: retail customers (walk-in / WhatsApp), schools, corporates, hospitality, church, security, government, resellers
- Core product: custom manufactured uniforms — branded, embroidered, sublimated, screen-printed
- Geographic base: South Africa (ZAR pricing, VAT 15%)
- Primary support channel: WhatsApp number connected to Meta Cloud API

## Key Capabilities

- ' Conversational AI-driven quotation flow
- ' School-specific uniform catalog
- ' Order lifecycle tracking (12 stages)
- ' PDF quotation generation & delivery
- ' Live consultant inbox takeover
- ' Bulk sizing list upload (CSV/Excel)
- ' Design approval workflow via WhatsApp
- ' Purchase order validation
- ' Lay-by scheme information
- ' SLA-tracked support tickets
- ' Broadcast messaging to client segments
- ' Role-based staff management

# 2. Architecture Overview

The platform follows a clean two-tier monorepo structure with a Node.js REST API backend and a React SPA frontend.

WhatsApp Client !' Meta Cloud API !' POST /webhook !' bot.service.js (state machine)  
Staff Browser !' React Dashboard !' Axios / REST !' Express API !' PostgreSQL (Supabase)  
Gemini AI !' ai.service.js !' FAQ answers, quotation info gathering

## Monorepo Structure

Root: npm workspaces with two apps:

- apps/api — Express.js REST API + WhatsApp bot engine
- apps/dashboard — Vite + React staff portal

### API Internal Layers

- src/index.js — Express app

bootstrap, middleware, route mounting

- src/routes/ — Route definitions (one file per resource)

- src/controllers/ — HTTP handler functions, input validation

- src/services/ — Business logic, DB queries, external API calls

- src/db/ — pg Pool, migrate.js (21 migrations), seed.js

- src/config/ — Centralised env config (required / optional helpers)

- src/middleware/ — requireAuth (JWT), requireRole (RBAC), upload (multer)

- src/utils/ — logger (winston), httpError, businessHours

### 3. Technology Stack

| Layer       | Technology                       | Purpose                          |
|-------------|----------------------------------|----------------------------------|
| Runtime     | Node.js v22 (ESM)                | Backend JavaScript runtime       |
| Framework   | Express.js 4                     | HTTP server & routing            |
| Database    | PostgreSQL via Supabase          | Primary data store               |
| DB Driver   | node-postgres (pg)               | PostgreSQL connection pool       |
| Auth        | jsonwebtoken + bcryptjs          | JWT signing & password hashing   |
| AI          | @google/genai (Gemini 2.5-flash) | FAQ answering, quotation parsing |
| Messaging   | Meta WhatsApp Cloud API v20      | WhatsApp send/receive            |
| PDF         | PDFKit                           | Quotation PDF generation         |
| File Upload | Multer                           | CSV/Excel bulk upload            |
| Excel/CSV   | exceljs + csv-parse              | Sizing list parsing              |
| Logging     | Winston                          | Structured JSON logs             |
| Security    | helmet + express-rate-limit      | HTTP headers + rate limiting     |
| CORS        | cors                             | Cross-origin policy              |
| Frontend    | React 18 + Vite                  | Staff dashboard SPA              |
| UI Library  | Ant Design 5                     | Component library                |
| State/Data  | TanStack React Query             | Server state + auto-polling      |
| Routing     | React Router v6                  | Client-side navigation           |
| HTTP Client | Axios                            | Dashboard !' API calls           |
| Deployment  | Render.com                       | Cloud hosting                    |

The project uses ESM (import/export) throughout — no CommonJS require() calls. All environment variables are centralised in apps/api/src/config/index.js using required() and optional() helpers that throw clearly on startup if a mandatory variable is missing.

## 4. Database Schema

PostgreSQL hosted on Supabase (transaction pooler, port 6543). Schema is managed via 21 sequential idempotent migrations in `src/db/migrate.js` — safe to re-run at any time.

### Enum Types

| Enum                  | Values                                                                                          |
|-----------------------|-------------------------------------------------------------------------------------------------|
| client_type           | retail, school, corporate, hospitality, church, security, government, reseller                  |
| order_stage           | 12 stages: quotation_requested !' quotation_submitted !' ... !' completed / cancelled / on_hold |
| session_state         | 45+ values tracking each step of the WhatsApp conversation state machine                        |
| quotation_status      | draft, sent, accepted, rejected, expired                                                        |
| staff_role            | admin, consultant, manager                                                                      |
| message_direction     | inbound, outbound                                                                               |
| message_status        | sent, delivered, read, failed                                                                   |
| ticket_category       | wrong_item, defective, missing_item, other                                                      |
| ticket_status         | open, in_progress, resolved, closed                                                             |
| purchase_order_status | pending_review, valid, invalid                                                                  |
| payment_status        | not_required, deposit_required, deposit_received, balance_outstanding,                          |
| payment_type          | deposit, balance, full                                                                          |

### Core Tables

| Table               | Key Columns                                                                         | Purpose                          |
|---------------------|-------------------------------------------------------------------------------------|----------------------------------|
| staff               | id, name, email, password_hash, role, is_active                                     | System users (admin/consultant/  |
| clients             | id, whatsapp_number, name, client_type, organisation, school_name, profile_complete | WhatsApp customers               |
| conversations       | id, client_id, assigned_staff, state (session_state), is_read_by_staff, sent_at     | WhatsApp sessions & bot state    |
| messages            | id, conversation_id, direction, body,                                               | All WhatsApp messages (in & out) |
| orders              | id, reference (BRQ-YYYYMMDD-XXXX), client_id,                                       | Manufacturing orders             |
| order_stage_history | id, order_id, from_stage, to_stage, changed_by,                                     | Full stage audit trail           |
| quotations          | notes, id, reference, client_id, line_items (JSONB),                                | Price quotations with PDF        |
| products            | id, category, name, sizes (JSONB), price,                                           | Product / catalog items          |
| school_catalog      | client_type, is_active, id, school_name, uniform_type, sizes (JSONB),               | School-specific uniform catalog  |
| broadcasts          | id, sent_by, message_type, body, recipient_count                                    | WhatsApp bulk messages           |
| purchase_orders     | id, quotation_id, po_number, status, validated_by                                   | Client PO submissions            |
| payments            | id, order_id, amount, payment_type, recorded_by,                                    | Deposit & balance payments       |
| order_size_entries  | received_at, id, order_id, person_name, size                                        | Bulk sizing (CSV upload)         |
| tickets             | id, client_id, category, description, status, sla_due_at                            | Complaints & returns             |
| sequence_counters   | key, year, value                                                                    | Auto-incrementing ref numbers    |

All tables have: UUID primary keys, TIMESTAMPTZ created\_at, updated\_at (auto-trigger)  
conversations.context (JSONB) stores conversation-specific state e.g. school lists, quotation history  
orders.reference format: BRQ-YYYYMMDD-XXXX (e.g. BRQ-20260702-1234)

## 5. WhatsApp Bot — Conversation Flows

The bot is a pure state machine. Every inbound WhatsApp message triggers `handleInbound()` in `bot.service.js`. The function reads `conversations.state` from the DB, routes through a switch statement, sends a reply via Meta Cloud API, and persists the new state back to PostgreSQL.

### Message Entry Point

- Meta sends a POST to `/webhook` with the message payload
- `webhook.controller.js` extracts: `phoneNumber`, `metaMessageId`, `body`
- `getOrCreateClientByWhatsapp()` finds or creates the client record
- `getOrCreateConversation()` finds the open session or creates a new one
- Message is saved to `messages` table, then state machine runs
- Outside business hours: acknowledgement sent but bot still processes (consultants see it next day)

### Intent Detection (`detectIntent`)

Before the state machine, the raw message body is classified:

| Intent Type | Trigger                     | Example                            |
|-------------|-----------------------------|------------------------------------|
| menu        | Single digit 1–7            | "2"                                |
| main_menu   | "0", "menu", "main menu"    | "menu"                             |
| order_ref   | Pattern BRQ-YYYYMMDD-XXXX   | "BRQ-20260702-1234"                |
| keyword     | Regex match on common words | "quote", "hours", "layby", "track" |
| freetext    | Anything else               | "I need 100 shirts please"         |

### Main Menu (state: `main_menu / new`)

- 1 !' Retail Customer !'  
retail\_menu state
- 2 !' School/Corporate/Bulk Orders !' corporate\_menu state
- 3 !' Request Quotation !'  
quotation\_requested (registration gate applies)
- 4 !' Track / Collect Order !'  
retail\_collection state
- 5 !' Branding & Embroidery Info !' `BRANDING_INFO` message (no state change)
- 6 !' Store Info & Contact !'  
`STORE_INFO` message (no state change)
- 7 !' Talk to Consultant !'  
awaiting\_consultant state

### Retail Customer Menu (state: `retail_menu`)

- 1 !' Product Pricing — fetches ALL products from DB, displays full list with prices by category



- 2 !' School Uniform Info — fetches distinct schools from school\_catalog, shows numbered list !' client picks school !' show uniforms + prices
- 3 !' Trading Hours — static hours message
- 4 !' Track/Collect Order — prompts for order reference
- 5 !' Lay-by Information — lay-by scheme details
- 6 !' Talk to Consultant !' awaiting\_consultant

## School Uniform Sub-flow

When option 2 (School Uniform Info) is selected:

- listDistinctSchools() queries school\_catalog for unique school names
- Bot sends numbered list e.g.  
"1. Greenside High  
2. Rosebank Primary..."
- State set to retail\_school\_select, schoolList stored in context JSONB
- Client replies with a number !' listUniformsForSchool() fetches items
- Bot sends: uniform type, description, sizes, price per item for that school

## School/Corporate/Bulk Menu (state: corporate\_menu)

- 1 ! Repeat Previous Order ! prompts for BRQ order reference ! creates new order with same details
- 2 ! New Uniform Development ! 4-step intake (garment ! sizes ! branding ! quantity) ! creates order ! awaiting\_consultant
- 3 ! Manufacturing Updates ! prompts for order reference ! fetches stage + progress %
- 4 ! Delivery Schedule ! prompts for order reference ! shows tracking info
- 5 ! Account Queries ! awaiting\_consultant
- 6 ! Dedicated Consultant ! awaiting\_consultant
- Note: Quotation is accessed from main menu option 3 or by typing "quote" at any time

## Quotation Flow (Multi-turn AI)

When a client requests a quotation (main menu 3 or keyword "quote"):

- Registration gate: if profile\_complete = false ! 3-question profile capture first (name, school/org, address)
- After gate: state set to quotation\_requested, AI greeting sent: "Hello [Name]! Please describe what you need a quotation for..."
- Client describes their need: "We need 300 crew-neck T-shirts in sizes S, M, L, school logo on chest"
- Gemini AI (gatherQuotationInfo) analyses history ! returns {status: "need\_more\_info", question: "..."} or {status: "ready", consolidatedRequest: "..."}
  - Bot asks up to 3 follow-up questions for missing: item types, quantities, sizes, branding details
  - After max 3 follow-ups OR when AI says ready ! createFromFreeText() ! generates quotation with line items
  - PDF sent to client via Meta Cloud API (sendDocument), reference e.g. BRQ-Q-00042
  - State ! awaiting\_consultant (consultant follows up to confirm)

## Registration Gate

Applies to quotation and new uniform development flows. If client.profile\_complete is false:

- State ! registration\_name, pendingIntent saved to context
- Asks: "What is your name?"
- State ! registration\_org\_or\_school: asks for school name (retail/school) or organisation (corporate)
- State ! registration\_address: asks for preferred store / physical address
- On completion: profile\_complete = true, resume from pendingIntent state

## Consultant Handover States

- awaiting\_consultant: bot has escalated, client messages are saved but bot is silent
- consultant\_active: staff member has taken over in dashboard, replies sent through dashboard go via WhatsApp API
  - Handback: staff clicks "Hand Back" ! state ! main\_menu, goodbye message sent to client, assignment cleared

## Design Approval Flow

Staff can trigger a design approval request from the dashboard. Client receives WhatsApp message:

- State ! corporate\_design\_approval, designApprovalOrderId stored in context
- Client replies: "APPROVE" ! recordDesignApproval(approved: true) ! stage advances ! main\_menu
- Client replies: "REJECT: reason" ! rejection reason logged ! awaiting\_consultant

## Support Ticket Flow

- Triggered by keywords: wrong item, defective, faulty, damaged, missing item, complaint
- State ! ticket\_category: client selects 1=Wrong Item / 2=Defective / 3=Missing / 4=Other
- State ! ticket\_description: client describes issue
- createTicket() creates ticket with SLA due date ! awaiting\_consultant

## Purchase Order Flow

- Triggered by keyword "purchase order" or "PO"
- State !' corporate\_po\_quotation\_ref: asks for quotation reference
- State !' corporate\_po\_number: asks for PO number
- submitPurchaseOrder() validates PO against quotation !' order created !' confirmation sent

## 6. API Endpoints

All API routes under `/api/` require JWT authentication (Authorization: Bearer <token>) except `/webhook` and `/api/staff/login`.

| Method | Path             | Access        | Description                               |
|--------|------------------|---------------|-------------------------------------------|
| POST   | /webhook         | Public        | Receive WhatsApp message from Meta        |
| GET    | /webhook         | Public        | Meta webhook verification (hub challenge) |
| POST   | /api/staff/login | Public        | Authenticate staff — returns JWT          |
| GET    | /api/staff       | Admin/Manager | List all staff members                    |
| POST   | /api/staff       | Admin         | Create new staff account                  |
| PATCH  | /api/staff/:id   | Admin         | Update staff details / role               |
| DELETE | /api/staff/:id   | Admin         | Deactivate staff account                  |
| GET    | /api/clients     | Auth          | List clients (filterable by type, search) |
| GET    | /api/clients/:id | Auth          | Client detail with order history          |
| PATCH  | /api/clients/:id | Auth          | Update client profile                     |

| Method | Path                            | Access | Description                                               |
|--------|---------------------------------|--------|-----------------------------------------------------------|
| GET    | /api/orders                     | Auth   | List orders (filter: stage, clientType, urgent, assigned) |
| POST   | /api/orders                     | Auth   | Create new order manually                                 |
| GET    | /api/orders/:id                 | Auth   | Order detail + stage history + payments                   |
| PATCH  | /api/orders/:id/stage           | Auth   | Advance order stage (sends WhatsApp notification)         |
| PATCH  | /api/orders/:id/assign          | Auth   | Assign order to staff member                              |
| POST   | /api/orders/:id/sizes           | Auth   | Upload CSV/Excel bulk sizing list                         |
| POST   | /api/orders/:id/design-approval | Auth   | Trigger design approval WhatsApp message                  |
| PATCH  | /api/orders/:id/payment         | Auth   | Record deposit or balance payment                         |
| GET    | /api/quotations                 | Auth   | List quotations                                           |
| POST   | /api/quotations                 | Auth   | Create quotation manually                                 |
| GET    | /api/quotations/:id             | Auth   | Quotation detail                                          |

| Method | Path                              | Access        | Description                                  |
|--------|-----------------------------------|---------------|----------------------------------------------|
| GET    | /api/quotations/:id/pdf           | Auth          | Stream quotation as PDF                      |
| PATCH  | /api/quotations/:id               | Auth          | Update quotation line items / status         |
| GET    | /api/products                     | Auth          | List products (filter: category, clientType) |
| POST   | /api/products                     | Admin/Manager | Create product                               |
| PATCH  | /api/products/:id                 | Admin/Manager | Update product                               |
| DELETE | /api/products/:id                 | Admin/Manager | Deactivate product                           |
| GET    | /api/purchase-orders              | Auth          | List purchase orders                         |
| PATCH  | /api/purchase-orders/:id/validate | Admin/Manager | Validate or reject a PO                      |
| GET    | /api/tickets                      | Auth          | List tickets (filter: status, category)      |
| GET    | /api/tickets/:id                  | Auth          | Ticket detail                                |
| PATCH  | /api/tickets/:id/status           | Auth          | Update ticket status / assign                |
| POST   | /api/broadcasts                   | Admin/Manager | Send WhatsApp broadcast to                   |

|       |                                 |      |                                                        |
|-------|---------------------------------|------|--------------------------------------------------------|
|       |                                 |      | client segment                                         |
| GET   | /api/broadcasts                 | Auth | List past broadcasts                                   |
| GET   | /api/conversations              | Auth | List conversations (filter: state, open, assignedToMe) |
| GET   | /api/conversations/unread-count | Auth | Returns {waiting: int, unreadMessages: int}            |
| GET   | /api/conversations/:id          | Auth | Conversation + all messages                            |
| POST  | /api/conversations/:id/reply    | Auth | Send WhatsApp reply as consultant                      |
| PATCH | /api/conversations/:id/takeover | Auth | Set state=consultant_active, assign to self            |
| PATCH | /api/conversations/:id/handback | Auth | Return to bot (state=main_menu), clear assignment      |
| PATCH | /api/conversations/:id/close    | Auth | Mark conversation closed                               |
| PATCH | /api/conversations/:id/assign   | Auth | Assign to another staff member                         |
| PATCH | /api/conversations/:id/read     | Auth | Mark all inbound messages as read                      |
| GET   | /api/school-catalog             | Auth | List school catalog entries (filter: school)           |

| Method | Path                        | Access        | Description                           |
|--------|-----------------------------|---------------|---------------------------------------|
| GET    | /api/school-catalog/schools | Auth          | List distinct school names            |
| POST   | /api/school-catalog         | Admin/Manager | Create catalog entry                  |
| PATCH  | /api/school-catalog/:id     | Admin/Manager | Update catalog entry                  |
| DELETE | /api/school-catalog/:id     | Admin/Manager | Delete catalog entry                  |
| GET    | /health                     | Public        | Health check — returns service status |



## 7. Staff Dashboard

The dashboard is a React 18 SPA built with Vite and Ant Design. It is served separately from the API (separate Render service or local port 5173). JWT is stored in localStorage (key: braq\_token). An Axios interceptor injects the Authorization header on every request and redirects to /login on 401.

### Pages & Features

| Page / Route     |               | Description                                                                                        |
|------------------|---------------|----------------------------------------------------------------------------------------------------|
| /                | Dashboard     | Overview stats: open orders, unread inbox, recent activity                                         |
| /inbox           | Inbox         | WhatsApp-style conversation list. Segments: Waiting / Active / All. 15s poll. Unread badge in nav. |
| /inbox/:id       | Chat          | WhatsApp bubble UI. Take Over / Hand Back / Close / Assign buttons. Reply                          |
| /orders          | Orders        | Filterable order list. Stage chip, urgency flag, assigned staff.                                   |
| /orders/:id      | Order Detail  | Full order card: stage timeline, size entries, payments, design approval                           |
| /clients         | Clients       | Client list with search, type filter. Customer number, registration status.                        |
| /clients/:id     | Client Detail | Profile card, order history, edit modal.                                                           |
| /quotations      | Quotations    | List with status, totals. Download PDF button. Send to client button.                              |
| /products        | Products      | Product catalog CRUD. Admin/Manager only create/edit/delete.                                       |
| /school-catalog  | School Cat.   | School uniform catalog CRUD. Filter by school. Add/edit entries with sizes, price, sort order.     |
| /purchase-orders | POs           | Purchase order list. Validate / reject actions for Admin/Manager.                                  |
| /tickets         | Tickets       | Support ticket list with SLA indicator (overdue highlighted). Status update.                       |
| /broadcasts      | Broadcasts    | Compose and send bulk WhatsApp messages to a client segment.                                       |

### Auth Flow

- POST /api/staff/login !' returns { token, staff: {id, name, role} }
- Token stored in localStorage as braq\_token, staff object as braq\_staff
- AuthContext (React context) reads from localStorage on mount
- ProtectedRoute redirects to /login if no token
- 401 response from any API call !' auto logout + redirect to /login

### Data Fetching Pattern

- TanStack React Query (useQuery) for all GET requests — automatic caching & stale management
- useMutation for POST/PATCH/DELETE — on success: invalidateQueries to refresh lists
- Inbox polling: conversations list every 15s, chat messages every 10s, unread count every 15s (nav badge)

## 8. School Catalog

The `school_catalog` table stores uniform items available for specific schools. This powers the WhatsApp bot's "School Uniform Info" flow where retail clients select their school and see available uniforms with prices.

### Data Model (`school_catalog` table)

| Column       | Type          | Description                                    |
|--------------|---------------|------------------------------------------------|
| id           | UUID          | Primary key                                    |
| school_name  | VARCHAR(200)  | School name (used as grouping key)             |
| uniform_type | VARCHAR(200)  | e.g. "Boys Summer Shirt", "Girls Winter Skirt" |
| description  | TEXT          | Optional additional description                |
| sizes        | JSONB         | Array of size strings e.g. ["S","M","L","XL"]  |
| price        | NUMERIC(10,2) | Price per item in ZAR                          |
| currency     | VARCHAR(3)    | Default: ZAR                                   |
| is_active    | BOOLEAN       | Soft delete / hide from bot                    |
| sort_order   | INT           | Display order within a school                  |

### Bot Flow Detail

- Client selects option 2 (School Uniform Info) from Retail menu
  - `listDistinctSchools()` `!' SELECT DISTINCT school_name WHERE is_active=true ORDER BY school_name`
    - Bot sends numbered list of schools
    - State = `retail_school_select`, `schoolList` saved in `conversations.context`
    - Client replies with number (e.g. "2") `!' schoolList[1] = selected school name`
      - `listUniformsForSchool(schoolName)` `!' all active entries for that school ordered by sort_order`
        - Bot sends formatted list: each item with uniform type, sizes, price
        - State resets to `main_menu`

### Dashboard Management

- School Catalog page at `/school-catalog` (admin & manager only for write operations)
  - Filter table by school name
  - Add entry: school name, uniform type, description, sizes (comma-separated), price, sort order
    - Edit: modal pre-populated with existing values
    - Delete: confirmation popconfirm before removing

## 9. Quotation System

## Auto-generation from Free Text (createFromFreeText)

When the AI determines the client has provided enough information, createFromFreeText() runs:

- Parses the consolidated description with Gemini AI !' structured line items array
  - Each line item: { description, quantity, unitPrice, total }
  - Matches against products catalog where possible
  - Creates quotation record with: reference (BRQ-Q-XXXXX), line\_items JSONB, subtotal, VAT (15%), total
  - Returns unmatchedText array for items that couldn't be auto-priced (consultant quotes manually)

## PDF Generation

- GET /api/quotations/:id/pdf !' streams a professional PDF using PDFKit
  - PDF includes: Braq Uni branding, client details, line items table, subtotal/VAT/total, validity date, payment terms
  - Sent to WhatsApp client via Meta Cloud API sendDocument (URL-based delivery)
  - URL format: https://[API\_BASE\_URL]/api/quotations/:id/pdf

## Manual Quotations (Dashboard)

- Staff can create quotations manually from /quotations page
  - Add/edit line items, set VAT rate, payment terms, lead time, validity date
  - Send to client button !' marks quotation as "sent", triggers WhatsApp notification
  - Status lifecycle: draft !' sent !' accepted / rejected / expired

## Reference Number Format

Orders: BRQ-YYYYMMDD-XXXX (e.g. BRQ-20260702-1234)

Quotations: BRQ-Q-XXXXX (e.g. BRQ-Q-00042)

Clients: BRQ-CUST-XXXX (e.g. BRQ-CUST-0015)

Sequence counters are per-year and auto-increment via sequence\_counters table

## 10. AI Integration

---

Google Gemini 2.5-flash is used via the @google/genai SDK. Two functions in ai.service.js:

### answerFaq(question, context)

- Used when a retail client asks a free-text question in retail\_pricing or retail\_layby states
- Context provided: full product list with prices, trading hours text, lay-by info text
- Gemini checks if the question can be answered from the provided context
- Returns: { type: "answer", text: "..." } or { type: "escalate", reason: "..." }
- If escalated: state !' awaiting\_consultant, consultant assigned message sent

### gatherQuotationInfo(history, context)

- Used during quotation\_requested state (multi-turn conversation)
- history: array of { role: "client"|"bot", text: string } — full conversation so far
- context: products list (for AI to know what can be matched)
- AI checks if the description has: item types, quantities per item, sizes/age range, branding requirements
- Returns: { status: "need\_more\_info", question: "What sizes do you need?" }
  - or: { status: "ready", consolidatedRequest: "300 crew-neck T-shirts, sizes S-XL..." }
- Structured JSON output enforced via Gemini responseSchema (type-safe)
- Graceful fallback: if AI fails !' returns ready status with whatever text was collected

### Gemini Configuration

- Model: gemini-2.5-flash-preview-04-17
- Output format: JSON enforced via responseType: "application/json" + responseSchema
- Temperature: not explicitly set (uses Gemini default)
- API key: GEMINI\_API\_KEY environment variable

## 11. Security & Authentication

---

### JWT Authentication

- POST /api/staff/login validates email + bcrypt password hash !' returns JWT (HS256)
- Token payload: { sub: staff.id, role: staff.role, iat, exp }
- JWT\_SECRET required in environment (must be strong random string in production)
- Token expiry: configurable (default 24h)
- requireAuth middleware: validates Bearer token on all /api/\* routes (except login)
- requireRole(...roles) middleware: checks token role against allowed roles

### HTTP Security

- helmet: sets secure HTTP headers (CSP, HSTS, X-Frame-Options, etc.)
- express-rate-limit: 200 requests per 15 minutes per IP on /api/\*
- CORS: production restricts origins to dashboard domain; dev allows localhost:5173 and :3001
- Webhook verification: Meta hub.challenge handshake for GET /webhook

### Passwords & Secrets

- Staff passwords hashed with bcryptjs (12 salt rounds)
- Default seed admin password: changeme123 — MUST be changed in production
- DATABASE\_URL, JWT\_SECRET, GEMINI\_API\_KEY, WHATSAPP\_TOKEN in .env (git-ignored)
- Supabase connection: transaction pooler with ?family=4 (IPv4 only) — avoids IPv6 issues

# 12. Deployment & Configuration

## Render.com Setup

- Two Render services: one for apps/api (Node web service), one for apps/dashboard (static site)
- Root: apps/api, Build: npm install, Start: node src/index.js
- Root package.json start script delegates to apps/api for Render compatibility
- Dashboard: Root apps/dashboard, Build: npm run build, Publish: dist/

## Required Environment Variables (API)

| Variable              | Description                                                       |
|-----------------------|-------------------------------------------------------------------|
| DATABASE_URL          | Full PostgreSQL connection string (Supabase pooler URL with ?     |
| JWT_SECRET            | Secret key for signing JWT tokens — long random string            |
| WHATSAPP_TOKEN        | Meta WhatsApp Cloud API bearer token                              |
| WHATSAPP_PHONE_ID     | Meta phone number ID                                              |
| WHATSAPP_VERIFY_TOKEN | Webhook verification token (set in Meta dashboard)                |
| GEMINI_API_KEY        | Google Gemini API key                                             |
| API_BASE_URL          | Public HTTPS URL of the API (for PDF links in WhatsApp documents) |

## Optional Environment Variables

| Variable            | Default           | Description                        |
|---------------------|-------------------|------------------------------------|
| PORT                | 3000              | API server port                    |
| NODE_ENV            | development       | "production" enables stricter CORS |
| SEED_ADMIN_EMAIL    | admin@braquni.com | Default seed admin email           |
| SEED_ADMIN_PASSWORD | changeme123       | Default seed admin password        |

## First-Time Setup Commands

```
npm install          # Install all workspace dependencies
npm run migrate      # Run all 21 DB migrations
npm run seed         # Create default admin account
npm run seed:catalog # (Optional) seed sample products into catalog
npm run dev          # Start API in watch mode (auto-restart on file changes)
cd apps/dashboard && npm run dev # Start dashboard dev server on :5173
```

# 13. Roles & Permissions

| Feature / Action                 | Admin | Manager | Consultant |
|----------------------------------|-------|---------|------------|
| View orders, clients, quotations | '     | '       | '          |
| Advance order stage              | '     | '       | '          |
| Reply in consultant inbox        | '     | '       | '          |
|                                  |       |         |            |

Take over / hand back conversations









Create/edit products







Create/edit school catalog









Validate purchase orders







Send broadcasts









Create/manage staff accounts







Delete products / catalog entries









Roles are enforced at both API level (requireRole middleware on routes) and dashboard level (UI elements hidden based on staff.role stored in localStorage).

## Default Seed Account

Email: admin@braquni.com

Password: changeme123

Role: admin

& Change this password immediately after first login in production!

---

Braq Connect™ Platform Documentation

— Generated 2 July 2026

This document is confidential and intended for internal use only. © Braq Uni.